

实验 L1 (10 分) 得分: _____

题号	11	12	13	14	15
解答	C	A	D	A	B

C 11. (2 分) 下面给出了 satMul2 函数的代码, 其中变量 `overflow` 在结果溢出时会被设置为 `0xFFFFFFFF`, 否则为 `0`。下面哪一个选项正确给出了 `overflow` 的表达式 (提示: 可根据 `x` 和 `2 * x` 的符号位来判断是否溢出):

- A. `(x ^ (x + 1)) >> 31;`
 B. `(x ^ (x + ~0)) >> 31;`
 C. `(x ^ (x << 1)) >> 31;`
 D. `(x ^ (x >> 1)) >> 31;`

```
/*
 * satMul2 - multiplies by 2, saturating to Tmin or Tmax if overflow
 *
 * Examples: satMul2(0x30000000) = 0x60000000
 *
 *           satMul2(0x40000000) = 0x7FFFFFFF (saturate to Tmax)
 *
 *           satMul2(0x90000000) = 0x80000000 (saturate to Tmin)
 */
```

```
int satMul2(int x) {
    int overflow, negative_x, raw_result, tmin, tmax;
    overflow = _____
    negative_x = x >> 31;
```

```
    raw_result = x << 1;
```

```
    10...0 → tmin = 1 << 31;
```

```
    01...1 → tmax = tmin + ~0;
```

```
    return (overflow & negative_x & tmin) | (overflow & ~negative_x &
    tmax) | (~overflow & raw_result);
}
```

ps. `overflow & negative_x` 充当掩码, 实现条件分支
从而 `return` 句等价于:

```
if (x < 0 && is_overflow)
    return Tmin;
if (x > 0 && is_overflow)
    return Tmax;
return raw_result;
```

*产生溢出的原因

- A. $x > 0$, 右移后最高位到了符号位变成 1, 溢出为负数
 B. $x < 0$, 右移后符号位变成 0, 溢出为正数

A 情况当且仅当 x 最高两位为 01

B 情况当且仅当 x 最高两位为 10

`overflow` 溢出情况下是 `0xFFFFFFFF`, 否则为 0, 从而为

C. `(x ^ (x << 1)) >> 31`; (利用算术右移)

A

12. (2分) 下面给出了 bitParity 函数的代码, 该函数用于判断整数 x 的二进制表示中 0 的个数是否为奇数。设整数 x 的二进制表示为 $x[31]x[30]\dots x[1]x[0]$, 则 $x1[15]$ 的值可表示为:

- A. $x[31] \wedge x[15]$
- B. $x[31] \wedge x[16]$
- C. $x[30] \wedge x[15]$
- D. $x[30] \wedge x[16]$

```
/*
 * bitParity - returns 1 if x contains an odd number of 0's
 * Examples: bitParity(5) = 0, bitParity(7) = 1
 */
int bitParity(int x) {
    int x1, x2, x3, x4, x5;
    16位 x1 = x ^ (x >> 16);
    8位 x2 = x1 ^ (x1 >> 8);
    4位 x3 = x2 ^ (x2 >> 4);
    2位 x4 = x3 ^ (x3 >> 2);
    1位 x5 = x4 ^ (x4 >> 1);
    return x5 & 1;
}
```

$x \gg 16: x[31]x[30] \dots x[16]$
 $\wedge x[31] \dots x[15]x[14] \dots x[0]$
 $\rightarrow x1[15] = x[31] \wedge x[15] \text{ (A)}$

ps. 0的个数奇偶性 = 1的个数奇偶性
 - 抑或运算可以让对应位置的两个1变成0
 通过不断“折叠” x , “消除”“成对的”1, 考察是否
 “剩下”“单独的”1, 得到结果。

D

13. (2分) 下面给出了 float_half 函数的代码, 该函数给出单精度浮点数 f 的一半 (即 $0.5 * f$) 的位模式。下面哪一个选项正确给出了 (1) 和 (2) 处应当填写的值?

- A. (1) 0x800000 (2) 8
- B. (1) 0x800000 (2) 23
- C. (1) 0x400000 (2) 8
- D. (1) 0x400000 (2) 23

```
/*
 * float_half - Return bit-level equivalent of expression 0.5*f for
 * floating point argument f.
 * Both the argument and result are passed as unsigned int's, but
 * they are to be interpreted as the bit-level representation of
 * single-precision floating point values.
 * When argument is NaN, return argument
```

```

*/
unsigned float_half(unsigned uf) {
    unsigned sign = uf >> 31;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFFFF;

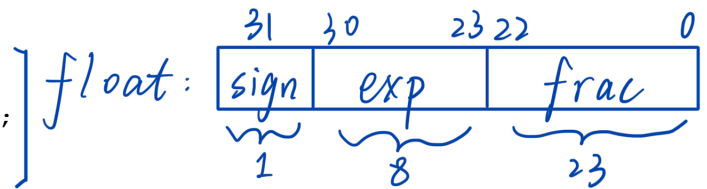
    /* Only roundup case will be when rounding to even */
    unsigned roundup = (frac & 0x3) == 3;

    if (exp == 0) {
        /* Denormalized. Must halve fraction */
        frac = (frac >> 1) + roundup;
    } else if (exp < 0xFF) {
        /* Normalized. Decrease exponent */
        exp--;

        if (exp == 0) {
            /* Denormalize adding back leading one */
            frac = (frac >> 1) + roundup + ____ (1) ____;
        }
    }

    /* NaN Infinity do not require any changes */
    return (sign << 31) | (exp << ____ (2) ____ ) | frac;
}

```



向偶数舍入：当且仅当末位11时会进位

case 1: 非规格化的小数
右移frac小数位（考虑舍入）

case 2: 规格化的小数
 $\text{exp}-1 \rightarrow x/2$

case 2.1: $\text{exp} == 1$
 $1 + \text{frac} \rightarrow 0.5 + \text{frac}/2$
 $\rightarrow \text{exp} = 0$, $\text{frac} >> 1$, frac首位设为1, 考虑舍入
 frac: 23位, 0x7FFFFFFF
 首位对应0x40000 (C/D)

case 3: 溢出：返回argument
 恢复sign, exp, frac的“位置”
 $\rightarrow \text{exp} << 23$ (D)

A

14. (2 分) 下面给出了 byteSwap 函数的代码，该函数交换后两个参数所指定的两个字节的数。下面哪一个选项正确给出了 (1) 和 (2) 处应当填写的内容？

- A. (1) 3 (2) ^
- B. (1) 8 (2) ^
- C. (1) 3 (2) |
- D. (1) 8 (2) |

```

/*
 * byteSwap - swaps the nth byte and the mth byte
 * Examples: byteSwap(0x12345678, 1, 3) = 0x56341278
 *            byteSwap(0xDEADBEEF, 0, 2) = 0xDEEFBEAD
 * You may assume that 0 <= n <= 3, 0 <= m <= 3
 */

```

```
int byteSwap(int x, int n, int m) {
    int n8 = n << ____ (1) ____;
    int m8 = m << ____ (1) ____;
```

n8, m8 → x需要左移的位数
 n=1 → 8, n=2 → 16, n=3 → 24
 从而n8 = n*8 = n << 3
 (A/B)

```
int swap_byte = ((x >> m8) ____ (2) ____ (x >> n8)) & 0xff;

return x ^ (swap_byte << n8) ____ (2) ____ (swap_byte << m8);
}
```

交换逻辑:

x = x ^ y;
 y = x ^ y;
 x = x ^ y;
 (A)

15. (2 分) 下面给出了 oddBits 函数的代码, 该函数将所有奇数位设置为 1 (设整数 x 的二进制表示为 x[31]x[30]...x[1]x[0], 奇数位是指 j 为奇数的所有 x[j])。下面哪个选项给出了 byte 的正确设置:

- A. 0x55
- B. 0xAA
- C. 0x11
- D. 0xff

设置一个掩码
 奇数位对应10101010, 即0xAA
 (B)

```
/*
 * oddBits - return word with all odd-numbered bits set to 1
 */
int oddBits(void) {
    int byte = ____;
    int word = byte | byte<<8;
    return word | word<<16;
}
```

实验 L4 (10 分) 得分: _____

题号	26	27	28	29	30
解答	D	B	D	B	D

D

26. (2 分) 在 ArchLab Part B 中, 我们需要在 SEQ 架构的基础上实现 iopq 指令。该指令的定义如下: iopq V, rB: 计算 rB op V, 并将结果存入寄存器 rB。下面是 SEQ 架构中的部分 HCL-rs 定义, 其中哪些位置需要添加 IOPQ?

A. (1) (2) (3)

B. (1) (3) (5)

C. (2) (3) (5)

D. (2) (4) (5)

Hint: IOPQ 相当于 OPQ+MRMOVQ

1. 取指:

icode:ifun = M1[PC]

rA:rB = M1[PC+1]

valC = M8[PC+2]

valP = PC+10

2. 译码:

valB = R[rB] (2)

3. 执行:

valE = valB OP valC (4)(5)

4. 仿存

5. 写回:

R[rB] = valE

6. 更新PC:

PC = valP

Source A: reg A 选哪个寄存?

u8 srcA = [

icode in { CMOVX, RMMOVQ, OPQ, PUSHQ } : ialign.rA; // (1)

icode in { POPQ, RET } : RSP;

true : RNONE;

]; destination E: 执行后的运算结果应放在哪? → reg B

u8 dstE = [

icode in { CMOVX } && cnd : ialign.rB;

icode in { IRMOVQ, OPQ } : ialign.rB; // (2)

icode in { PUSHQ, POPQ, CALL, RET } : RSP;

true : RNONE;

]; input A to ALU: ALU 的第一个参数 → valC

u64 aluA = [

icode in { CMOVX, OPQ } : reg_read.valA; // (3)

icode in { IRMOVQ, RMMOVQ, MRMOVQ } : ialign.valC; // (4)

icode in { CALL, PUSHQ } : NEG_8;

icode in { RET, POPQ } : 8;

]; ALU function → ifun

u8 alufun = [

icode == OPQ : ifun; // (5)

true : ADD;

];

B

27. (2分) 在 ArchLab Part B 中, 我们将架构从 SEQ 一步步改进到 pipe_std 这个标准的五级流水线架构。下列说法中错误的一项是?

- A. 随着架构的一步步改进, 关键路径的长度逐渐变短, 时钟频率可以逐渐提高 ✓
- B. 从 pipe_s2 演化到 pipe_s3a 的过程中引入了数据冒险, 可通过增加寄存器解决 ✗ 转发
- C. pipe_s4a 将架构中的执行阶段分离出来, 使内存访问和 ALU 计算可以并行执行 ✓
- D. 标准的五级流水线 pipe_std 具有分支预测, 可能发生分支预测错误 ✓

为了马上确定下一条指令的位置

D

28. (2分) ArchLab Part C 中, 我们需要优化 ncopys 代码。下面两个代码片段截取自某两个同学的 ncopys, 它们均使用标准的五级流水线架构。下列关于这两段代码性能的说法, 正确的一项是?

- A. 代码 S 性能更优, 因为将访问内存的指令集中在了一起, 可以有效增加缓存命中率 ✗ 无效
- B. 代码 S 性能更优, 因为将功能相似的指令如 iaddq 集中在一起, 便于流水线优化 ✗ 无效
- C. 代码 T 性能更优, 因为将部分公共指令提前执行, 使每个循环中平均指令数量变少 ✗ 无区别
- D. 代码 T 性能更优, 因为将 iaddq 指令插入到了前面, 避免了加载/使用冒险的发生

代码 S	代码 T
Loop: mrmovq (%rdi), %r10 rmmovq %r10, (%rsi) andq %r10, %r10 jle Skip iaddq \$1, %rax Skip: iaddq \$8, %rdi iaddq \$8, %rsi iaddq \$-1, %rdx jg Loop	Loop: mrmovq (%rdi), %r10 iaddq \$8, %rdi rmmovq %r10, (%rsi) iaddq \$8, %rsi andq %r10, %r10 jle Skip iaddq \$1, %rax Skip: iaddq \$-1, %rdx jg Loop

加载/使用冒险

D

29. (2分) 在 ArchLab Part C 中, 我们需要同时优化代码 ncopys 和架构 ncopys, 评分标准基于 $c = cpe + 2 * ac$ 。关于这一部分的优化策略, 下列说法中正确的一项是?

- A. 应该优先优化 cpe, 因为 ac 的权重不大, 且 cpe 不依赖于架构的设计 ✗
- B. 应该优先优化 ac, 因为这样可以提高时钟频率, 降低执行与调试的时间 ✗
- C. cpe 和 ac 相互独立, 优化 cpe 不会影响 ac, 优化 ac 也不会影响 cpe ✗
- D. 在 ncopys 中添加更多的流水线阶段可以降低 ac, 但可能使 cpe 增加

eg. 增加 IOPQ 可以显著降低 CPE

CPE 优化空间更大 (实践出真知??)

ac: average cycle time 时钟周期时间 (硬件)

cpe: cycles per element (软件)

软硬件协同工作, 相互耦合

B

30. (2分) 在 ArchLab Part C 中, 某同学准备在标准的五级流水线架构 pipe_std 的基础上进一步优化。该同学输入命令 ./target/debug/ysim --arch pipe_std -I, 得到了下面的结果。该同学进行下列哪一项优化, 可以在降低 ac 的同时, 不使 cpe 显著增加?

- A. 在架构中增加 iopq 指令, 提高部分指令的执行效率
- B. 在架构中缩短 e_cnd, d_bubble 等信号的路径长度
- C. 在架构中执行和访存阶段中间增加“设置条件码”阶段
- D. 在架构中将设置条件码从执行阶段改到访存阶段进行

用于合并指令, 降低 CPE $\times$ ac 甚至由于增加复杂度而提升.

五级变六级流水线
 $\rightarrow$ 更多 hazard $\Rightarrow$ 显著增加 CPE
 更高分支预测错误代价.

延迟 CC 生成 $\Rightarrow$ 回路变长
 propagate order:

lv.1: w_stall m_icode d_dstM m_dstM prog_stat mem_write d_srcB e_dstM
 m_stall m_valE d_dstE w_dstM e_ifun e_stat e_icode d_stat f_pc aluB
 mem_read e_valA m_dstE alufun w_bubble d_icode w_valE e_stall aluA
 mem_data f_bubble d_srcA mem_addr d_valC d_ifun w_dstE w_valM prog_term
 d_stall f_stall

lv.2: imem alu dmem reg_file f_align f_icode f_ifun e_valE m_valM
 m_stat d_rvalA d_rvalB instr_valid need_regids need_valC m_bubble
 set_cc f_stat

lv.3: ialign pc_inc reg_cc f_rA f_rB f_valC f_valP cc f_pred_pc

lv.4: cond e_cnd d_bubble e_bubble e_dstE d_valA d_valB
 dependency graph visualization is generated at:
 pipe_std_dependency_graph.html

Lv.1 Lv.2 Lv.3 Lv.4

register $\rightarrow$ ALU $\rightarrow$ CC $\rightarrow$ e_cnd

缩短 Lv.4 信号的关键路径长度 $\Rightarrow$ 设置更短的时钟周期 $\Rightarrow$ ac $\downarrow$

不改变指令执行机制 $\rightarrow$ CPE 不变.